Oral exam handbook

Group 15 • Final assessment preparation • 5 November 2026

Practise each answer in your own words, then answer the follow-up without reading. A strong response states a decision, explains the mechanism, points to evidence, and names the relevant limit. Every question can go to any group member.

Requirements and domain behavior

1 What problem does the system solve

It observes laboratory experiments, preserves their temperature history, and notifies required temperature transitions. The room temperature is a complete-sensor mean. Temperature control is another system's responsibility. The scientific value is traceability and timely awareness of excursions; this service does not decide whether an experiment is scientifically valid.

Follow-up: Identify a missing alert, an incorrect mean, and a slow history query as three different failures.

2 Why do we wait for all sensors

The requirement defines temperature as the mean of all allocated sensors for one measurement. Using only the sensors that arrived first changes the quantity. We group by experiment and measurement ID and verify timestamp/hash agreement. Partial readings are durable so replacement workers can finish the same group.

Follow-up: What happens when two of three readings arrive before an executor restarts?

3 Why not use the most recent value from each sensor

Those values can belong to different times. Their mean may never have represented the physical space at any sampling instant. Our grouping preserves measurement identity instead of silently carrying forward old values. A missing sensor delays that group rather than being filled with an invented value.

Follow-up: Explain the effect on both correctness and the ten-second target.

4 What happens during stabilization

Complete groups are monitored and an entry into the inclusive allowed range triggers Stabilized. Going outside and re-entering can trigger another Stabilized. This does not start carry-out automatically; the researcher sends the start event. Stabilization aggregates may be persisted internally but are excluded from required carry-out history responses.

Follow-up: Why distinguish stored internal data from the public query contract?

5 When do we send OutOfRange

Only for the first outside complete mean in a continuous running excursion. Consecutive outside points all belong in outside-history queries but do not each need an alert. Returning in range resets the transition condition, so a subsequent outside point creates a new alert.

Follow-up: How many alerts and outside points result from outside, outside, inside, outside?

6 Are range and time boundaries inclusive

Yes. A temperature exactly equal to either configured threshold is inside. A query includes points exactly at its start or end time, including equal bounds. The snapshot cache uses left and right binary searches to preserve these semantics and duplicate-timestamp ordering.

Follow-up: Why would using the same binary-search side for both bounds be wrong?

7 What can be queried immediately after configuration

Configuration and status are available after the write is acknowledged. An empty history is a valid response before carry-out points exist. Termination does not delete history. A nonexistent experiment and an existing experiment with no points must be distinguished by the API contract.

Follow-up: Find this distinction in the public API/integration tests.

8 What if a sensor or phase event is invalid

The engine rejects unallocated sensors, conflicting duplicates, inconsistent timestamps/hashes, invalid phase order, and transitions with incomplete groups. Kafka input errors are stored in quarantine so the partition can progress. Quarantine preserves evidence; it does not repair the experiment or make its missing data correct.

Follow-up: Why is a rising rejection count a problem even if Kafka lag falls?

9 What is the difference between outside points and intervals

The required outside endpoint returns every outside carry-out measurement. The convenience interval endpoint groups excursions and ends them at the next inside point or termination. An active excursion has no end yet. Neither endpoint's row count is the number of notification delivery attempts.

Follow-up: Can a successful retry change the historical outside points?

10 What do we do with the measurement hash

We check that sensors in a group agree and forward the original opaque value as notification `cipher_data`. We do not decrypt it or derive temperatures from it. The original notification validator uses its own keys and producer truth to check correctness; our independent sink is a different test double.

Follow-up: Why is passing the original validator stronger contract evidence than only using our sink?

Sources: engine, models, engine tests, assignment.

Architecture and scaling decisions

11 Why separate these services

They have different obligations: intake preserves order, executors calculate, Data commits and serves history, the gateway exposes routes, and the notifier tolerates receiver delays. Separation allows targeted replacement/scaling. Its cost is more network calls and operations; on one VM a modular monolith could be simpler and cheaper to execute.

Follow-up: Which component should scale if only external notification delivery becomes slow?

12 Why Python

The work includes substantial network/database waiting, so asynchronous I/O is useful. Python also makes the state machine and validation easy to inspect. Suitability is established by the measured workload, not by claiming the language is universally fast. CPU-bound processing and shared-host contention still require measurement.

Follow-up: Why would changing language not automatically fix an overloaded database?

13 What is stateless about an executor

It has temporary load counters and connections but owns no authoritative long-lived experiment state. Partial groups, prior range status, versions, and completed identities are loaded from Data. A replacement can recompute a task. The tradeoff is extra persistence/network work compared with keeping experiments permanently in worker RAM.

Follow-up: Where is a two-of-three-sensors partial group after a successful task?

14 Why PostgreSQL and Redis together

PostgreSQL supplies authoritative transactions, uniqueness, indexes, and concurrency control. Redis caches disposable serialized query results. Redis-only authority would need another durability and atomicity strategy. Redis loss increases database work but does not erase the experiment history; recovery testing exercises that fallback.

Follow-up: Why did the recovery test also restart Data after adding the RAM cache?

15 Why keep JSONB state and relational measurements

JSONB conveniently represents evolving state, including partial sensor groups. Relational measurement columns and indexes support ordered time queries and identity uniqueness. This is a deliberate hybrid. Putting everything into an unindexed JSON document would make query and constraint behavior harder to control.

Follow-up: Name one uniqueness constraint and one history index.

16 Why Caddy when api.py exists

The deployed gateway forwards public queries directly to Data and configuration to an executor, reducing Python proxy overhead. The Python API module still defines the development interface/OpenAPI. Internal commit and notification-claim routes remain off the public gateway. Both routes and error behavior need contract tests.

Follow-up: Which service actually reads PostgreSQL for a public history query?

17 Why least-busy scheduling

It is simple and appropriate for broadly comparable batches. Reported active work plus local reservations avoid assigning every task to a worker whose reported load is stale. Worker capacity remains enforced. Batches can differ in sensors and database work, so this is not an exact prediction of completion time.

Follow-up: What protects the executor if the scheduler's load observation is old?

18 Why did we deploy one executor

The earlier same-host matrix found one executor achieved 98.077 HTTP/s against 100 offered, while two/four achieved about 93.4–93.6. Additional workers competed for shared resources. One was the smallest passing configuration. That historical result explains the choice but should be remeasured on new hardware or changed workloads.

Follow-up: Distinguish a scalable worker abstraction from a measured capacity gain.

19 What does the autoscaler actually do

It is a host-side opt-in policy, not continuously enabled in the demo. Defaults are one to four replicas, scale-up after five seconds of queue above 1,024 or occupancy at least 75%, and scale-down after sixty seconds fully idle. It cannot distinguish every downstream bottleneck.

Follow-up: Why might scaling up during CPU saturation reduce HTTP throughput?

20 What limits concurrent tasks

One active task per Kafka partition preserves order. With P partitions, K executors and capacity C, the bound is min(P, K×C). At 16 assigned partitions and one capacity-eight executor, it is eight. This is a concurrency bound; batch service time and skew determine throughput.

Follow-up: Why can one hot partition still limit progress with many idle workers?

Sources: scheduler, discovery, autoscaler, gateway, replica matrix.

Consistency and failures

21 What is the most important transaction

The Data transaction commits state versions, measurements, notification obligations, rejected input, and the partition checkpoint together. It prevents durable progress from claiming that work is done while history or an alert obligation is missing. Kafka acknowledgement follows this commit.

Follow-up: What would break if the outbox job were created only in notifier RAM?

22 Why store a database checkpoint as well as Kafka offsets

Kafka and PostgreSQL do not commit atomically together. The database checkpoint identifies effects already durable if Kafka acknowledgement is lost. Replay can safely skip them and later advance Kafka. Logical measurement identities also prevent duplicate aggregates when completed measurements reappear at new offsets.

Follow-up: Which duplicate case is not solved by comparing only Kafka offsets?

23 What if we acknowledge Kafka first

A crash before the database commit could leave Kafka pointing past events whose history and notification jobs do not exist. Ordinary consumption would skip those effects. Our order is durable application commit, then Kafka next-read offset. The remaining replay window is handled by deduplication.

Follow-up: What offset does Kafka store after durably processing offset 45?

24 What prevents concurrent writers from losing updates

Commit proposals carry expected state versions and checkpoint. Data checks them inside the transaction. A conflicting version aborts the whole proposal, and the executor reloads/recomputes. Deterministic state-write ordering also reduces conflicting lock order. It is not acceptable to merge arbitrary partial results without checking their assumptions.

Follow-up: Describe the stale-middle-of-three integration regression.

25 Why was batching safe

It reduced sequential state-write round trips while preserving sorted lock order and checking every returned version. A mismatch still rolls back all effects. The regression test proves all-or-nothing behavior when one middle update is stale, then verifies successful progress after the version is corrected.

Follow-up: Why is checking only the first returned update insufficient?

26 What happens to duplicate sensor messages

An identical repeated reading within a pending group occupies the same sensor entry; it is not counted twice. A conflicting value is rejected. Completed measurement identities suppress re-emission of completed aggregates/jobs, including logical replay at new offsets. These are distinct duplicate cases with distinct checks.

Follow-up: Where are completed identities retrieved for a task?

27 What if an earlier sensor never arrives

Its incomplete group prevents later complete groups for that experiment from draining in order. Pending-group limits prevent unbounded accumulation, and invalid transitions are rejected. This protects correctness but can prevent timeliness. The product needs an explicit missing-sensor policy before choosing interpolation, timeout completion, or partial means.

Follow-up: Why would silently averaging available sensors change the requirement?

28 Are notifications exactly once

Database jobs are unique and durable. External acceptance has a separate ambiguity: the receiver can accept before the sender loses its response or crashes. Retrying can duplicate acceptance. We send an idempotency key, but exactly-once external effect requires the receiver to honor it.

Follow-up: Why does a PostgreSQL transaction not eliminate that remote crash window?

29 What do notification leases guarantee

They temporarily assign a pending job and allow retry after expiry. Result updates require the current lease token, preventing a stale owner from acknowledging another owner's work. The fifteen-second lease supports recovery but may exceed the ten-second latency target after a crash; recovery correctness is not an outage SLA guarantee.

Follow-up: Why is an arbitrarily tiny lease also risky?

30 What happens when Redis or PostgreSQL fails

Redis loss falls back to authoritative SQL after bounded cache failure handling. PostgreSQL loss prevents new durable commits, so the scheduler cannot safely acknowledge those events; backlog/retries grow. The tests establish specific restart recovery cases. They do not provide cross-host redundancy or a guarantee through indefinite outages.

Follow-up: Which symptoms distinguish cache failure from loss of the authority?

Sources: Data, executor, notifier, schema, recovery evidence.

Performance and experimental reasoning

31 What is our strongest current performance result

The original tools' native variable run used 100 main experiments, ten sensors, and one-second samples. All five observed HTTP target phases passed, all 703 expected alerts arrived, maximum delay was 2.884 seconds, and 202 complete history comparisons passed. It is a specified local result, not the hosted professor's grade.

Follow-up: Which exact target rates were observed, and did that sequence include 200?

32 Can we sustain 200 requests per second

The native fixed target passed our local rule at 190.573 achieved/s, with a maximum alert delay of 8.958 seconds. The independent live-history 200 peak failed at 150.329/s. Different pacing, query timing, history state, and receiver paths matter. The evidence does not support an unrestricted 200-rps claim.

Follow-up: What is the native fixed run's margin above our local throughput threshold?

33 Why do notification counts differ between native runs

The original simulator produces random temperature paths. The number of range transitions, and therefore required alerts, differs even with the same experiment/sensor/rate settings. Each run must compare received with its own expected set. Equal alert counts are not a prerequisite for comparable configured workloads.

Follow-up: Why is comparing only received counts weaker than expected-versus-received validation?

34 What does 202 history checks mean

Two full history endpoints were checked for 101 completed experiments: 100 main experiments plus one warmup completed before the main concurrency. It is not 202 concurrent experiments, and it is separate from the thousands of continuous HTTP validator responses.

Follow-up: Why does completing warmup first matter for the 100-concurrent-experiment claim?

35 Why is p99 below ten seconds insufficient

A strict deadline can be violated by a small tail that a percentile hides. Missing alerts may not appear in a latency histogram at all. We need required/received counts, validation correctness, and every accepted alert below ten seconds. Exactly ten fails the new predicate and deadline-miss counter.

Follow-up: How would a dashboard look if no alerts were received at all?

36 What does keeping up with HTTP mean in our tests

We compare valid completions with offered rate, track errors and omitted requests, and include drain time. The local threshold is 95% of target; the rubric does not prescribe that percentage. Queue growth and tail latency still matter, especially when a short run only narrowly meets the local rule.

Follow-up: Why cannot a target-rate gauge alone prove throughput?

37 Why use an open-loop test as well as the original generator

Independently scheduled requests continue to represent offered demand when responses slow. A closed-loop client can reduce its own effective offered load while waiting. Our open-loop result records intended/scheduled/completed counts, planned-time latency, and phase drain; a bounded pending queue fails explicitly if it omits work.

Follow-up: Explain coordinated omission without relying on the term itself.

38 How can zero HTTP errors still be a failure

The independent 200 peak intended 6,000 requests but scheduled 5,526. They all succeeded, yet 474 offers were omitted and completions drained beyond thirty seconds. Achieved rate was 150.329/s, below 190. Correct responses are necessary, but completing fewer requests late does not establish capacity.

Follow-up: Calculate 5,526 divided by approximately 36.759 seconds.

39 Did we statistically prove the optimizations caused the gain

No. We have before/after observations, mechanism-level reasoning, regression tests, and diagnostic measurements. Native random traces and shared-host conditions differ, and changes were not all isolated. Repeated controlled runs with matched inputs would strengthen causal attribution. Failed and reverted trials remain in the evidence.

Follow-up: Name one confounder and one practical way to control it.

40 Which workloads remain unproven

Arbitrary long histories, prolonged load/credit behavior, all variable event-arrival schedules, every partition skew, and the professor's external workload are not established by short local passes. Capacity runs also preceded the final scheduler fairness revision. Repeat representative tests on that exact revision in a separate test window.

Follow-up: Why is a longer soak relevant even if memory caching is bounded?

Sources: native variable, native fixed, independent failure, phase accounting, validation.

Optimizations and bottleneck diagnosis

41 What does the new history cache save

For small histories it retains a versioned ordered snapshot, allowing many different time windows to avoid repeated SQL/Redis work. Binary search selects inclusive bounds. Histories above 512 points fall back to indexed SQL/Redis. The cache is bounded by 16,384 points and 256 entries, so arbitrary experiment duration cannot fill RAM indefinitely through this cache.

Follow-up: Why probe 513 points instead of returning the first 512?

42 Why do metadata and history come from one SQL statement

Separate statements could pair an older version with newer rows or vice versa under concurrent writes. One statement gives a consistent database snapshot for both. The cache then associates its content with the correct version. This is a correctness condition, not merely an optimization detail.

Follow-up: What happens when a request needs a version newer than a cached snapshot?

43 Why is commit-time metadata warming fenced

Writer A can commit, release its connection slowly, and resume after writer B has already committed. Without a generation check, A could publish older metadata last. Active-writer and generation fences allow only the final uncontended writer to warm known committed values. A forced-interleaving integration test protects this case.

Follow-up: Why is checking only whether A's SQL commit succeeded insufficient?

44 Why not horizontally scale Data immediately

Its local metadata/snapshot coherence assumes one process owns all writes. Separate processes can retain different versions without shared invalidation. Adding Uvicorn workers is also multiple processes. We would first introduce a shared version/invalidation protocol, remove local metadata caching, or redesign reads, then test concurrency and failure behavior.

Follow-up: Why does having Redis somewhere in the architecture not automatically coordinate RAM caches?

45 What did monitoring itself do to performance

The recovered outbox was large, and repeated historical counts scanned substantial data. We indexed status and slowed historical sent/rejected refresh to once per minute while retaining faster pending/failed/age checks. Monitoring should detect deadline problems without competing unnecessarily with the work it observes.

Follow-up: Which dashboard totals can now be slightly stale, and why is that acceptable?

46 How do pool wait and occupancy differ

Pool wait measures time to acquire a connection. Occupancy measures time while holding it, including transaction/SQL work, application work, and scheduling. High wait may indicate a busy pool, but enlarging it can increase database contention. Neither metric alone is a pure SQL execution profile.

Follow-up: What extra evidence distinguishes slow SQL from event-loop scheduling pressure?

47 Why optimize the test controller

It shared the two CPUs with the service. Repeated Compose inspection was much more expensive than direct container inspection. Caching IDs and inspecting both load containers together reduced that overhead while keeping workload and validation. A test driver that consumes capacity can distort the service's measured boundary.

Follow-up: Why should a dedicated load-generator host still record its own saturation?

48 Why did we revert the raw transport and uvloop trial

It did not improve the independent boundary workload: its busy phase achieved about 131.367/s versus 150.329 in the snapshot run, and still omitted offered work. These are unreplicated observations, but there was no evidence to retain extra complexity. The failed result remains archived and is not described as final code.

Follow-up: What experiment would justify reconsidering that change?

49 What problem did per-partition backpressure solve

A hot partition could consume the global queue budget and pause intake for other partitions despite spare worker capacity. Per-partition high/low watermarks let other partitions continue, with the global guard retained. The queue count includes in-flight work. It improves fairness; it does not parallelize the hot partition itself.

Follow-up: At 16 partitions and default settings, calculate the per-partition high watermark.

50 How would you identify two different bottlenecks

If CPU, HTTP delay, and scheduler backlog rise together, inspect shared process/generator CPU and compare at fixed workload. If histories are current but outbox age and 503s rise, inspect the receiver/network. The first can benefit from reducing compute/coordination work; the second needs delivery-path action, not more executors.

Follow-up: State what observation would falsify each initial hypothesis.

Sources: Data, history, cache/race tests, controller measurements, reverted trial, scheduler tests.

Operations, costs, and assessment

51 What was wrong with course Kafka connectivity

The provider's broker certificate names did not match the public DNS name used by the demo. The demo overlay disables hostname matching while retaining CA trust and client-certificate authentication. This is a specific compatibility exception and weakens hostname identity checking; valid public-name certificates should restore strict matching.

Follow-up: Distinguish TCP reachability, TLS identity, client authentication, and partition assignment.

52 What topic and group do we use for the professor

The demo uses `experiment` and `group15-temperature-demo-v1`, with the external course notification endpoint. The original-tool self-test uses a local topic/group/receiver. The modes share the application database, so stopping generators alone does not switch the application back to demo configuration.

Follow-up: Why should pending outbox jobs be inspected before switching receivers?

53 Which dashboard parts are ours

The operational dashboard on loopback 3004 is ours. The local course dashboard on 3009 uses the instructor panels and local datasources. We added the bottom panel named Latest local test, driven by our explicit checks. A pinned old local success is not the hosted professor's current result.

Follow-up: What would you check before interpreting a green panel as evidence?

54 What did the Docker cleanup remove

Killing stopped processes; removing containers removed their writable layers; removing images removed executable layers/references. Named volumes survived the listed commands. PostgreSQL later reopened successfully, so preservation is now demonstrated more strongly than merely listing volume names. Disk/VM loss still needs a real off-host backup strategy.

Follow-up: Why is pruning volumes dangerous after all containers have been removed?

55 What did backlog recovery establish

The existing group had 2,860,538 retained offsets behind the initial sampled broker positions. We processed retained work without resetting to latest and eventually reached sampled positions with zero sampled lag. Old alerts remained late. Broker-expired events are outside what this catch-up can reconstruct.

Follow-up: Why is the scheduler's queue of zero not the same as a complete broker lag measurement?

56 How much does the deployment cost

The recorded estimate is one VM plus provisioned disk: hourly VM price plus disk quantity times monthly price divided by 730. With the recorded rates and unconfirmed 20 GiB gp3 assumption it is about $0.043211/hour. Credits, IPv4, traffic, tax, discounts, and course infrastructure are excluded; it is not the bill.

Follow-up: Why should four containers on this VM not be billed as four VMs?

57 Why does disk still need a plan

At 100 experiments and 1 Hz, uninterrupted carry-out creates 8.64 million aggregate rows per day, before indexes, jobs, and other storage. Bounded cache memory does not bound persistent history. Retention limits cover some disposable infrastructure, but required history needs capacity, export/archival policy, and tested backups.

Follow-up: Why is a same-disk backup insufficient for disk loss?

58 What do the 92 tests prove

They cover specific correctness, API, cache-race, scheduler, validation, and recovery-related behaviors; fourteen use integration infrastructure. They do not prove all inputs, indefinite runtime, all failures, or hosted workload performance. The load/recovery artifacts provide separate kinds of evidence. This report preparation did not rerun workloads or change application code.

Follow-up: Choose one regression test and state the failure it would catch.

59 How do we aim for Excellent

Show a healthy working demo, correct data and strict alerts under the applied workload, and HTTP capacity with honest phase accounting. Explain distinct responsibilities and identify multiple bottlenecks using evidence. Train every member across the whole system. More report pages or containers cannot replace working behavior and individual understanding.

Follow-up: Which remaining experiment most directly strengthens our capacity claim?

60 What did you personally contribute

Answer with your real task, the decision you made, a file/result/review that substantiates it, and how you verified it. Code, experiments, operations, analysis, and documentation can all be meaningful contributions. Suggested study roles are not authorship evidence. Follow the course's applicable disclosure rules for tools and assistance.

Follow-up: Explain one component outside your preparation specialty without handing off the answer.

Sources: demo configuration, readiness, cost evidence, rubric, team rehearsal.

Worked exercise A Calculate the observable behavior

Use three sensors, range 25–26 inclusive, and complete matching groups. Assign running samples M1–M5 timestamps 10–14 respectively, followed by termination at 15.

Phase and sample	Sensor values	Mean	Notification
Stabilizing S1	20, 21, 22	21	None
Stabilizing S2	24, 25, 26	25	Stabilized
Running M1	24, 26, 28	26	None
Running M2	26, 27, 28	27	OutOfRange
Running M3	27, 28, 29	28	None

Phase and sample	Sensor values	Mean	Notification
Running M4	24, 25, 26	25	None
Running M5	22, 23, 24	23	OutOfRange

Expected outputs: five carry-out history points; three outside points M2, M3, M5; two carry-out notifications and one Stabilized. Query bounds [11,12] return M2 and M3. Excursion intervals are 11 to 13 and 14 to 15; the end identifies return-in-range or termination, not an assertion that the closing point is outside.

Remove sensor C from M2. No M2 mean is available; M3 waits behind it even if complete. Duplicate A with the same value adds no second contribution. Duplicate A with a conflicting value is rejected. Each member should derive all four variations independently.

Worked exercise B Trace the two crash boundaries

Start with application checkpoint 40. A task processes 41–45, completes a measurement, and creates a job. Data commits 45; the response is lost. Replay finds durable progress and does not create a second measurement/job. Kafka can acknowledge next-read offset 46. If the transaction had rolled back, checkpoint 40 would instead permit recomputation.

Now move the crash to after the external receiver accepts but before local result acknowledgement. Database input progress cannot resolve that remote outcome. Retry may repeat acceptance. State why idempotency belongs at the receiver and why the lease token only protects local job ownership.

Worked exercise C Interpret load honestly

A 200/s phase lasts thirty seconds. It expects 6,000 offers, schedules 5,526, receives 5,526 valid responses, and drains at 36.759 seconds. Achieved rate is approximately 150.329/s. It fails both complete offering and the local 190/s rate threshold despite zero response errors.

A native 200/s run instead achieves 190.573/s with zero validator errors. It passes that local threshold narrowly. Explain why these different workloads cannot be combined into a claim that the final architecture sustains every 200/s request pattern.

Worked exercise D Diagnose before changing capacity

Observation	Hypothesis to test	Evidence that would redirect you
CPU high, intake/HTTP backlog rises	Shared compute/driver contention	CPU mostly idle but database pool waits dominate
Histories current, old pending jobs, receiver 503s	External delivery bottleneck	Receiver fast but Data claim/result requests are slow
Free executor slots, one lagging partition	Key skew or sequential hot partition	Balanced partition lag with all Data operations slow
SQL recovery appears successful with Redis off	Persistent store served the query	The response was supplied by an un-cleared Data snapshot cache

For each row, name a metric, a source function, and one targeted experiment. Do not diagnose from one graph without a competing explanation.